



Dance4Life

Envelhecimento activo

- Agentes e Sistemas MultiAgentes -

Mestrado em Inteligência Artificial

Carlos Bergueira

PG11605

Diego Silva

PG59999

Filipa Pereira

PG42201

Rui Rodrigues

PG 7942

Universidade do Minho | Junho 2026

Keywords

Sistemas Multiagente

SPADE

FIPA-ACL

Envelhecimento Ativo

Human Activity Recognition (HAR)

Tecnologia, Metodologia e Standards

Python

SPADE

API REST

XMMP

jsonpikle

Flask

API OpenWeatherMap

Firebase

Machine Learning

Reinforcement Learning

Android / Smartwatch

Sensores físicos

Sensores virtuais

FIPA-ACL

Ontology

Privacy by Design

UUIDs

Agent UML

PowerBI

[D3.js](#)

Leaflet

Agentes

Características

Autonomia: atua sem intervenção direta do utilizador ou de outros agentes

Reatividade: percebe alterações no ambiente e responde a essas mudanças

Proatividade: toma iniciativas para atingir os seus objetivos, não se limitando a reagir

Capacidade social: comunica e coopera com outros agentes

Racionalidade: escolhe ações que maximizam a probabilidade de atingir os seus objetivos

Persistência: mantém-se ativo ao longo do tempo, monitorizando o ambiente

Adaptabilidade: pode ajustar o seu comportamento perante novas situações

Agentes

BackgroundAgent

BaseSenderAgent (Flask)

SensorAgent

CoordinatorAgent

HARAgent

EnvironmentAgent

DatabaseAgent

ClusteringAgent

Sistemas Multiagentes

Características

Distribuição: o problema é dividido por vários agentes.

Cooperação: os agentes trabalham em conjunto para atingir objetivos comuns.

Comunicação: existe troca de mensagens entre agentes.

Coordenação: as ações dos agentes são organizadas para evitar conflitos.

Descentralização: normalmente não existe uma entidade central que controla todo o sistema.

Escalabilidade: é possível adicionar novos agentes sem alterar significativamente a arquitetura.

Robustez: a falha de um agente não implica necessariamente a falha do sistema.

Flexibilidade: o sistema adapta-se facilmente a novas necessidades ou condições.

Arquitectura do Dance4Life



Camada 1 — Aquisição no *edge* *smartphone* e *smartwatch*

Acelerómetro
3 eixos (x, y, z)

Giroscópio
Rotação angular

Freq. cardíaca
PPG óptico (bpm)

GPS
Lat / Lon contínuo

Camada 2 — Pré-processamento local

Magnitude inercial
Norma L2 do vetor

Indicadores de ritmo
Cadência, energia, variabilidade

Observações compactas
Feature vector reduzido
(sem séries brutas)

REST

Camada 3 — Ingestão e serviços de contexto (Flask)

Receção de atividade
/activity (POST)
métricas derivadas

Contexto ambiental
/environment?lat&lon
meteo · eventos

Recomendações
/recommend (POST)
resultado do pipeline

Camada 4 — Orquestração multiagente (SPADE)

Sensor
Agent

Coordinator
Agent

HAR
Agent

Environment
Agent

Database
Agent

Clustering Agent

Camada 5 — Persistência e analítica

Firebase
Eventos · matching
· recomendações

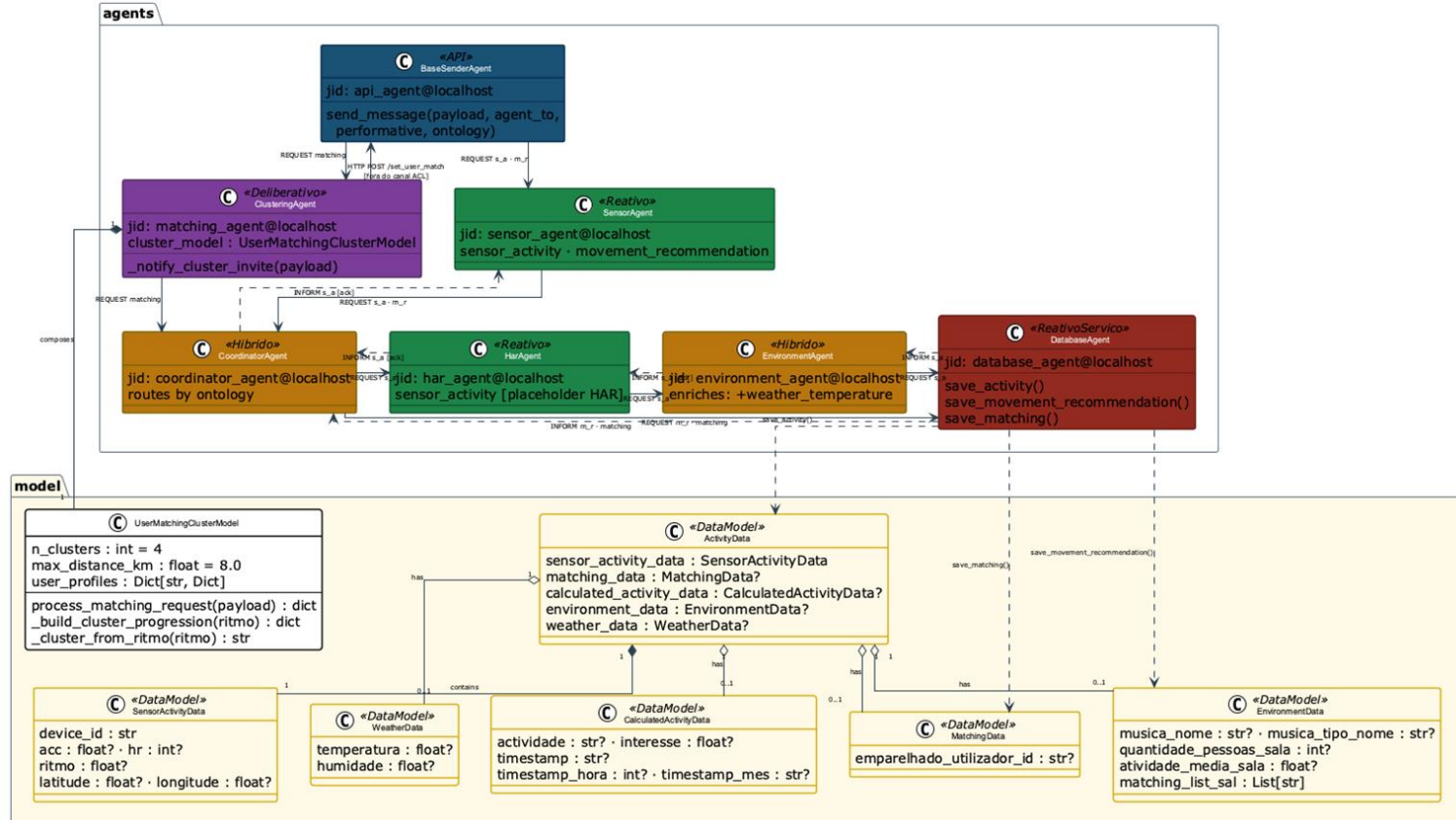
Dashboards analíticos
Análise longitudinal

Privacidade
Minimização de dados
sem séries brutas

— Fluxo principal

- - - - - Ramo paralelo

Diagrama de Classes



Implementação

sender

ontology

performative

conversation_id

payload

—

XMPP

FIPA-ACL

```
async def setup(self):  
    await super().setup()  
    self.add_behaviour(self.ReceiveDatabaseDataBehaviour())
```

```
# -----  
# SETUP BASE  
# -----  
async def setup(self):  
    # register uma vez  
    self.add_behaviour(self.RegisterBehaviour())  
  
    # heartbeat loop  
    heartbeat = self.HeartbeatBehaviour(period=10)  
    self.add_behaviour(heartbeat)
```

```
class DatabaseAgent(BaseBackgroundAgent):  
  
    class ReceiveDatabaseDataBehaviour(CyclicBehaviour):  
        async def run(self):  
            msg = await self.receive(timeout=10)  
  
            if msg:  
                sender = msg.sender  
                performative = msg.get_metadata("performative")  
                ontology = msg.get_metadata("ontology")  
                conversation_id = msg.get_metadata("conversation-id")  
  
                try:  
                    payload = jsonpickle.decode(msg.body)  
  
                    if ontology == AgentOntologies.SENSOR_ACTIVITY:  
                        if performative == AgentPerformatives.REQUEST:  
                            try:  
                                payload.pop("visited_agents", None)  
  
                                await save_activity(payload)  
  
                                await self.agent.forward_message(  
                                    behaviour=self,  
                                    payload=payload,  
                                    agent_to=sender,  
                                    performative=AgentPerformatives.INFORM,  
                                    ontology=AgentOntologies.SENSOR_ACTIVITY,  
                                    conversation_id=conversation_id  
                                )  
                            except Exception as e:  
                                print(f"[DatabaseAgent] Erro ao enviar mensagem para DatabaseAgent: {e}")  
  
                        elif performative == AgentPerformatives.INFORM:  
                            print("[DatabaseAgent] INFORM recebido - dados de atividade processados")
```


Diagrama de Colaboração: Matching

Dance4Life -- Diagrama de Colaboracao AUMl Protocolo FIPA-Request :: *matching*

FIPA-Request Interaction Protocol

Despacho paralelo apos `asyncio.sleep`
(mesmo pedido POST `/collect_data_activities`)
REQUEST ACL (→): canal XMPP/SPADE
HTTP (→): efeito lateral fora do canal ACL
INFORM (→): confirmacao de persistencia
conv-id: conv-{jid}-guid-{uuid}

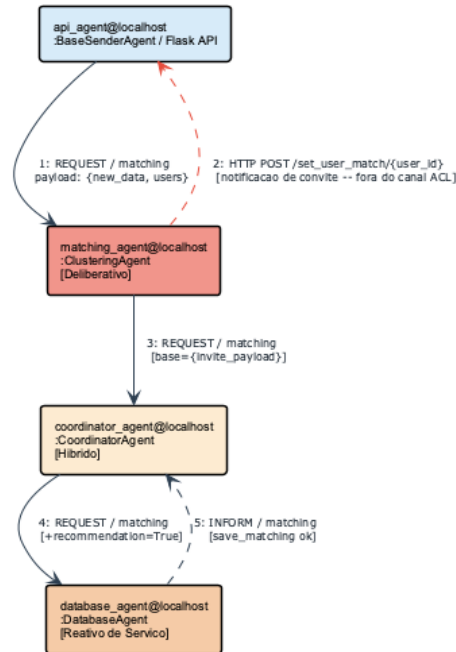


Diagrama de Colaboração: Movement Recommendation

Dance4Life -- Diagrama de Colaboracao AUMML Protocolo FIPA-Request :: *movement_recommendation*

FIPA-Request Interaction Protocol

Fluxo simplificado: sem passagem por HarAgent/EnvironmentAgent

REQUEST (-->): encaminhamento

INFORM (...): confirmacao de persistencia

conv-id: conv-{jid}-guid-{uuid}

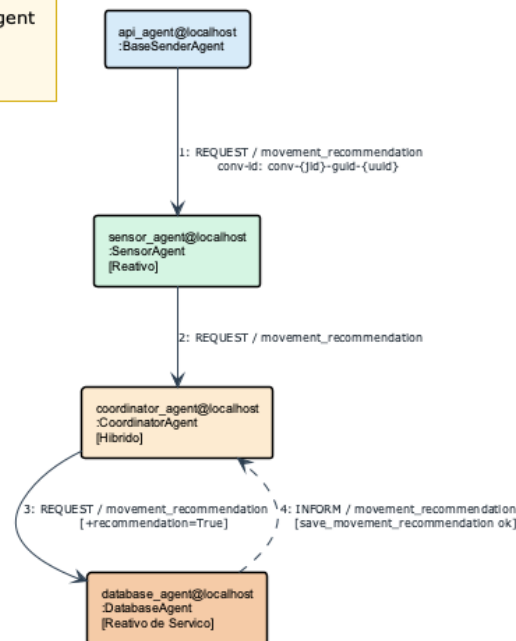


Diagrama de Colaboração: Sensor Activity

Dance4Life -- Diagrama de Colaboracao AUMI Protocolo FIPA-Request :: *sensor_activity*

FIPA-Request Interaction Protocol

REQUEST (-->): encaminhamento de dados
INFORM (...): confirmacao local (ack)
conv-id: conv-{jid}-guid-{uuid}
agree/failure definidos em config
mas nao aplicados sistematicamente

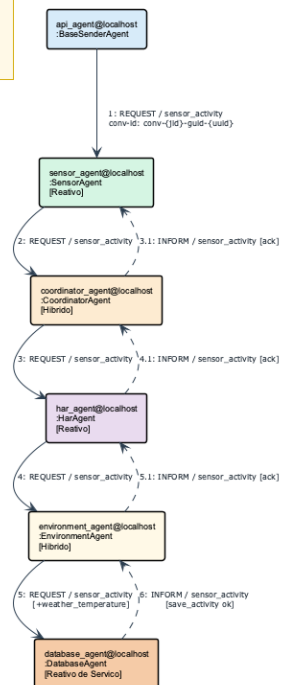


Diagrama de Atividades: Matching

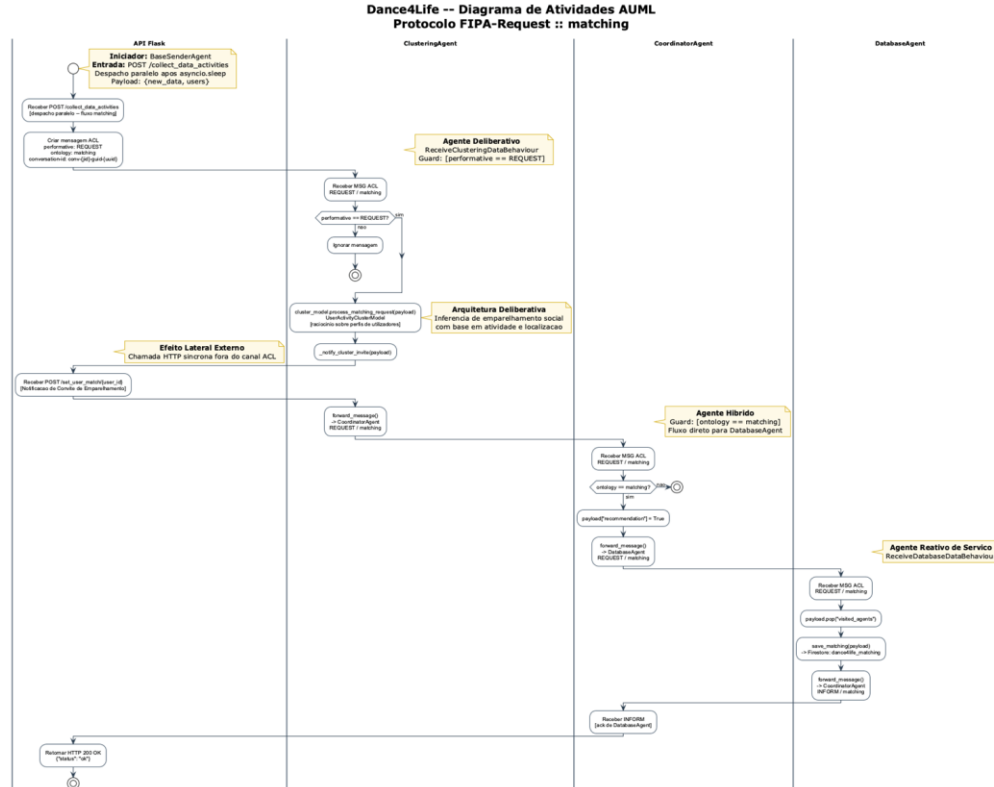


Diagrama de Actividades: Movement Recommendation

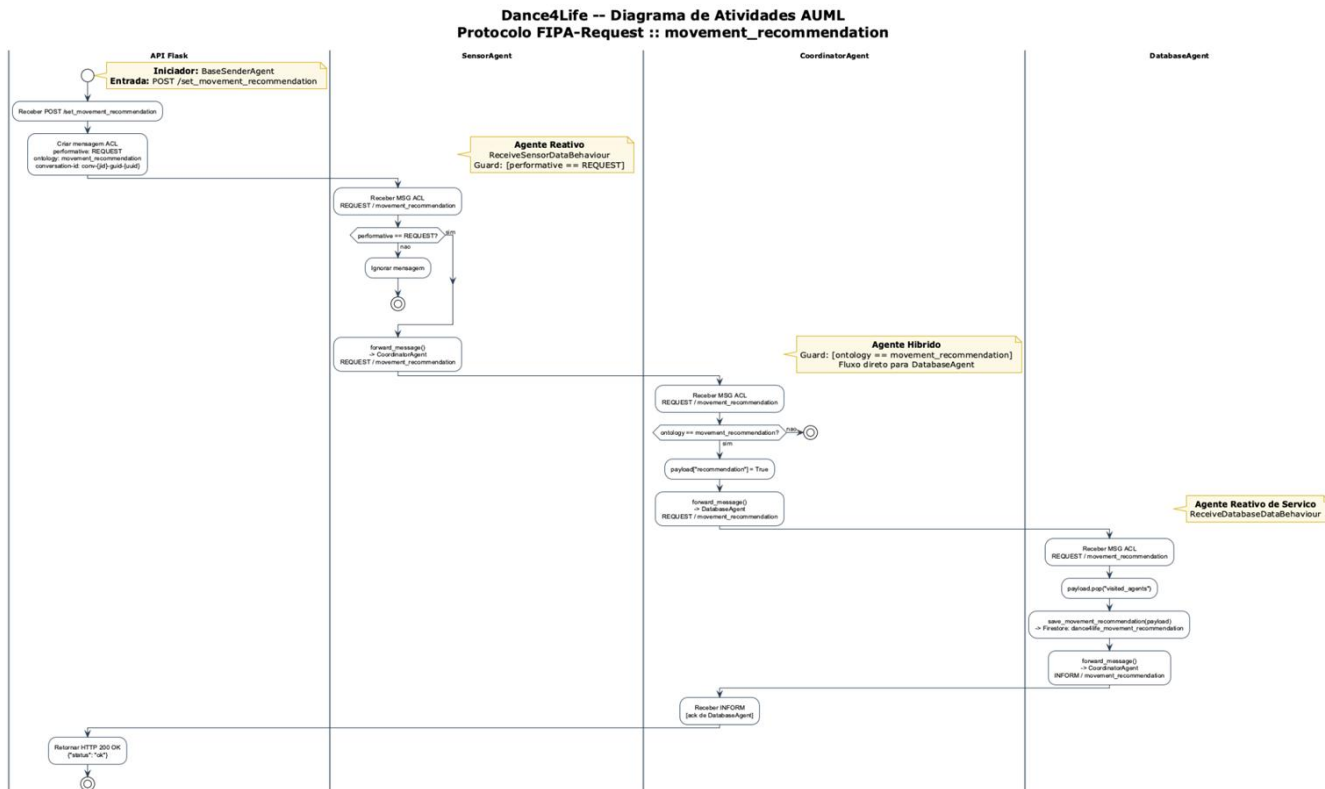


Diagrama de Actividades: Sensor Activity

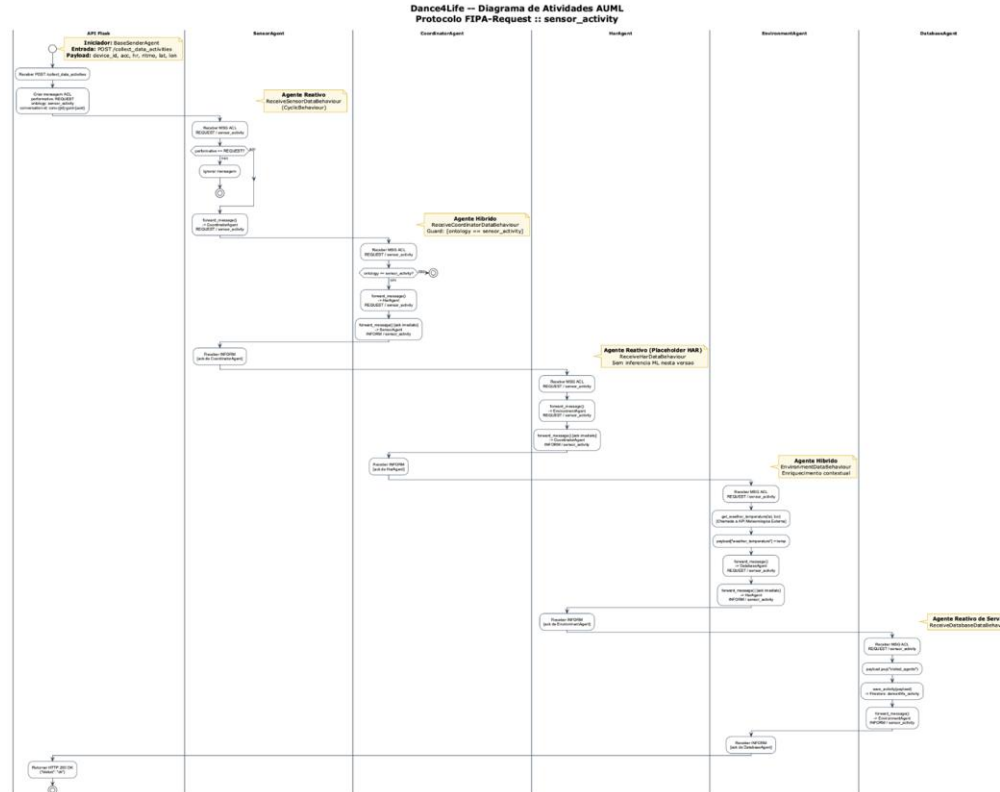


Diagrama de Sequência: Matching

Dance4Life -- Diagrama de Sequencia AUMI
Protocolo FIPA-Request :: matching

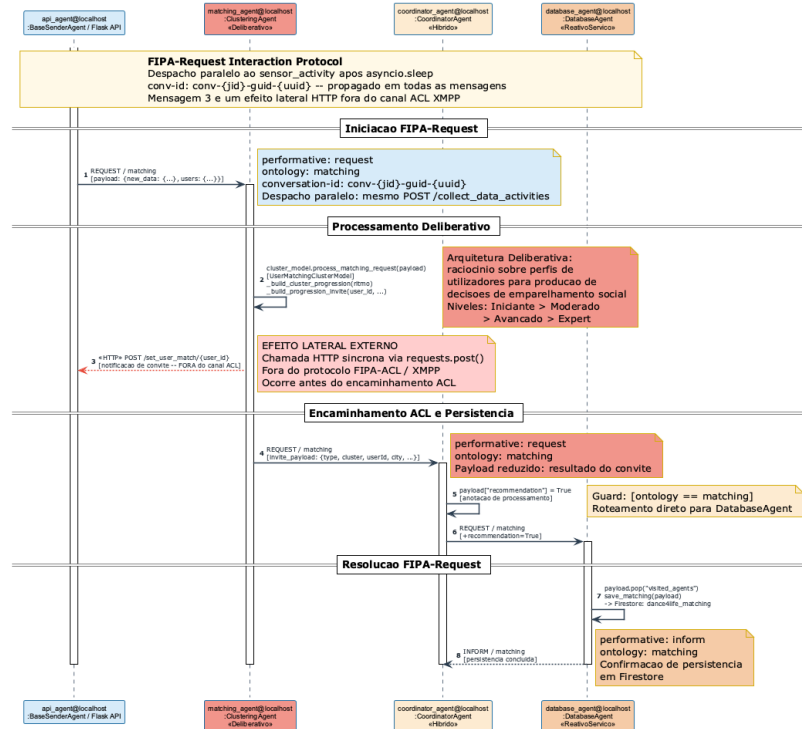


Diagrama de Sequência: Movement Recommendation

Dance4Life -- Diagrama de Sequencia AUMI Protocolo FIPA-Request :: *movement_recommendation*

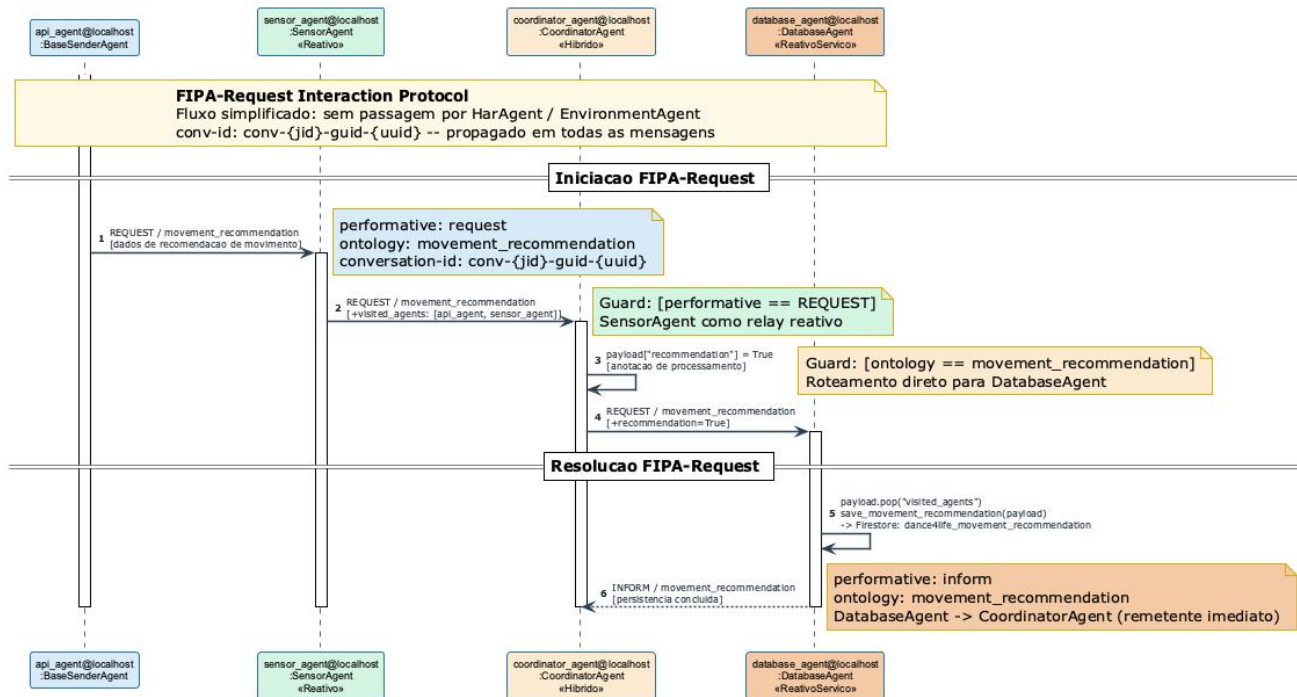


Diagrama de Sequência: Sensor Activity

Dance4Life -- Diagrama de Sequencia AUMI Protocolo FIPA-Request :: *sensor_activity*

